

Resource Routes

In Laravel, you can define a resource route that maps all the CRUD operations for a resource to controller methods. This is a convenient way to define routes for a resource without having to manually define each route, which is what we have been doing so far.

Resource Naming Convention

When creating a resource route, Laravel follows a naming convention for the controller methods.

Verb	URI Pattern	Controller Method	Description
GET	/jobs	index	Display a listing of the resource
GET	/jobs/create	create	Show the form for creating a new resource
POST	/jobs	store	Store a newly created resource in storage
GET	/jobs/{job}	show	Display the specified resource
GET	/jobs/{job}/edit	edit	Show the form for editing the specified resource
PUT/PATCH	/jobs/{job}	update	Update the specified resource in storage
DELETE	/jobs/{job}	destroy	Remove the specified resource from storage

We have been using the naming convention in our examples so far, but let's take a closer look at our routes:

php artisan route:list

```
GET|HEAD  /
.....
.....
GET|HEAD  jobs
.....
..... JobController@index
POST      jobs
.....
..... JobController@store
GET|HEAD  jobs/create
.....
.... JobController@create
GET|HEAD  jobs/{id}
.....
..... JobController@show
GET|HEAD  up
.....
.....
```

As you can see, we are following the convention.

Creating A Resource Route

We are now going to delete our entire `JobController` class and file and create a new one using the `--resource` flag. Delete the file and run the following command in your terminal:

```
php artisan make:controller JobController --resource
```

This will re-create a file named `JobController.php` in the `app/Http/Controllers` directory. Open this file and you will see that it contains a class definition that extends the base controller class. It also contains all the methods that we need for a resource route.

So any time that you want to create a resource route, you can use the `--resource` flag with the `make:controller` command.

Let's re-add the logic that we had previously. I also like to add a comment with a description and the route/method that the method is handling. Here is what the `JobController` class should look like so far:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class JobController extends Controller
{
    // @desc    Show all jobs
    // @route   GET /jobs
    public function index()
    {
        $title = 'Available Jobs';
        $jobs = [
            'Software Engineer',
            'Web Developer',
            'Data Scientist',
        ];

        return view('jobs/index', compact('title', 'jobs'));
    }

    // @desc    Show create job form
    // @route   GET /jobs/create
    public function create()
    {
        return view('jobs.create');
    }

    // @desc    Store a new job
    // @route   POST /jobs
    public function store(Request $request)
    {
```

```
        $title = $request->input('title');
        $description = $request->input('description');

        return "Title: $title, Description: $description";
    }

    // @desc    Show a single job
    // @route    GET /jobs/{id}
    public function show(string $id)
    {
        return "Showing job $id";
    }

    // @desc    Show the form for editing a job
    // @route    GET /jobs/{id}/edit
    public function edit(string $id)
    {
        return "Edit job $id";
    }

    // @desc    Update a job
    // @route    PUT /jobs/{id}
    public function update(Request $request, string $id)
    {
        return "Update job $id";
    }

    // @desc    Delete a job
    // @route    DELETE /jobs/{id}
    public function destroy(string $id)
    {
        return "Delete job $id";
    }
}
```

We only have two methods that show a view. The rest are just returning strings. We will update all of this stuff soon.

Using The Resource Controller In A Route

Now that we have created our resource controller, we can use it in a route. Open the `routes/web.php` file and create a new resource route that uses the `JobController` controller:

```
Route::resource('jobs', JobController::class);
```

So we don't have to add any other routes for our jobs CRUD operations. This single line of code will create all the routes that we need for our jobs resource.

You can visit `/jobs`, `/jobs/create`, `/jobs/1`, `/jobs/1/edit`, etc. in your browser to see the results.

Adding New Routes To A Resource Controller

You can also add new routes to a resource controller. Let's say that you want to add a new route to show the form for creating a job. You can add a new route to the `JobController` class as you normally would. We aren't going to keep this method, but I just want to show you how to add a new method to a resource controller.

```
// @desc    Save a job to favorites
// @route   POST /jobs/{id}/save
public function save(Job $job): string
{
    return 'Save Job';
}
```

Then, you can add a new route to the `routes/web.php` file like this:

```
Route::get('/jobs/{id}/save', [JobController::class, 'save'])->name('jobs.save');
```

MAKE SURE that you add this BEFORE the `Route::resource('jobs', JobController::class);` line. If you add it after, it will not work. It will use the `show` method instead of the `save` method.

You can keep this route and method, but we won't use it until much later.